Enhancing Linear Constraint Base Presolve by Exploiting Clique Structures

Zhao-Wei Wang, Chang-Long Li, Wei-Kun Chen, Yu-Hong Dai
wangzhaowei@lsec.cc.ac.cn, lichanglong@bit.edu.cn, chenweikun@bit.edu.cn, dyh@lsec.cc.ac.cn

Motivation

- Clique structures describe the pair-wise conflict relationship between a set of binary variables, with a wide range of applications [Borndörfer, 1998; Fügenschuh and Martin, 2005].
- Domain propagation, redundant constraint detection and coefficient strengthening relies on approximation bounds to improve the formulation of MIP problems [Savelsbergh, 1994; Achterberg, 2007; Achterberg et al., 2019]
- Cliques can be utilized to attain such bounds, but we have to add some cliques as constraints to ensure an equivalent transformation.

  ▷ **increase the number of constraints!**

Contribution

- We propose a clique partition algorithm **with source attribution** to obtain a clique partition using only a "slice" of the full clique table.
- Based on it, we propose clique-enhanced presolve techniques based on domain propagation, redundant constraint detection and coefficient strengthening, **without increasing the number of constraints**.
- Numerical experiments demonstrate that the three proposed techniques can effectively improve the performance of the open source MIP solver HiGHS.

Preliminary & Challenges

Domain Propagation

For the following r-th constraint

$$a_{rj_0}x_{j_0} + \sum_{j\neq j_0} a_{rj}x_j \leq b_r,\ a_{rj_0} > 0, r \in \mathcal{M},$$

where $\mathcal{M} := \{1, \ldots, m\}$ is the index set of constraints. We can derive valid bounds for x_{j_0}:

$$x_{j_0} \leq \max\left\{b - \sum_{j\neq j_0} a_{rj}x_i \mid x \in \mathcal{X}\right\} / a_{j_0}, \qquad (1)$$

where $\mathcal{X}$ is the feasible region of the MIP problem

$$\mathcal{X} := \{x \mid Ax \leq b,\ \ell \leq x \leq u,\ x_{\mathcal{I}} \in \mathbb{Z}^{|\mathcal{I}|},\ \mathcal{I} \subseteq \mathcal{N}\},$$

and $\mathcal{I}$ is the index set of integer variables: $\forall\ j \in \mathcal{I}$, $x_j \in \mathbb{Z}$.

- **Clique structures can be utilized to tighten the right-hand side of (1)**:

$$\begin{aligned} \max\quad & z := b - \textstyle\sum_{j\neq j_0} a_{rj}x_j \\ \text{s. t.}\quad & \textstyle\sum_{j\in C_q} x_j \leq 1,\ \{C_q\}_{q=1}^{Q} \subseteq \mathcal{C},\ Q \in \mathbb{N}_+, \end{aligned}$$

where $\mathcal{C}$ is the set of cliques valid for $\mathcal{X}$ (clique table)

$$\mathcal{C} := \{C \subseteq \mathcal{N} \mid \sum_{j\in C} x_j \leq 1 \text{ is valid for } \mathcal{X}\}.$$

Redundant Constraint Detection

Consider the following r-th constraint:

$$\sum_{j\in\mathcal{N}} a_{rj}x_j \leq b_r, r \in \mathcal{M},$$

we can remove it from the problem and keep the feasible domain $\mathcal{X}$ unchanged if $z_r \leq b_r$, where z_r is defined as

$$\begin{aligned} \max\quad & z_r := \textstyle\sum_{j\in\mathcal{N}} a_{rj}x_j \\ \text{s. t.}\quad & \textstyle\sum_{j\in\mathcal{N}} a_{ij}x_j \leq b_i,\ \forall\ i \in \mathcal{M}\setminus\{r\}, \\ & \ell \leq x \leq u,\ x_{\mathcal{I}} \in \mathbb{Z}^{|\mathcal{I}|},\ \mathcal{I} \subseteq \mathcal{N}. \end{aligned} \qquad (2)$$

▷ Feasible domain of (2): $\mathcal{X}_r\ (\supseteq \mathcal{X})$

▷ The set of cliques that are valid for $\mathcal{X}_r$: $\mathcal{C}_r\ (\subseteq \mathcal{C})$

- **Clique structures in $\mathcal{X}_r$ can be utilized to approximately solve (2):**

$$\begin{aligned} \max\quad & \hat{z}_r\ (\geq z) := \textstyle\sum_{j\in\mathcal{N}} a_{rj}x_j \\ \text{s. t.}\quad & \textstyle\sum_{j\in C_q} x_j \leq 1,\ \{C_q\}_{q=1}^{Q} \subseteq \mathcal{C}_r,\ Q \in \mathbb{N}_+. \end{aligned}$$

- **However, there are no efficient algorithms to separate the cliques in $\mathcal{C}_r$ from $\mathcal{C}$.**
- **Therefore, we need to add the cliques that are (partially) implied by the r-th constraint to ensure an equivalent reduction.**

Coefficient Strengthening

Consider the following r-th constraint:

$$\sum_{j\notin C_0} a_{rj}x_j + \sum_{j\in C_0} a_{rj}x_j \leq b,\ C_0 \subseteq \mathcal{N} \qquad \text{(original)}$$

Assumptions ($\star$):

▷ $a_{rj} \geq 0,\ \forall\ j \in C_0$ ▷ $\sup\left\{\sum_{j\notin C_0} a_{rj}x_j\right\} \leq b$

▷ $\sup\left\{\sum_{j\notin C_0} a_{rj}x_j\right\} + \max_{j\in C_0}\{a_rj\} - b := \Delta > 0$

Then constraint (original) can be strengthened with a non-negative slack $f(x_C) : \{0,1\}^{|C|} \mapsto \mathbb{R}_+$, where

▷ $f(0, \ldots, 0) = \Delta$

▷ $f(x_j = 1,\ x_{C\setminus\{j\}} = 0) = \max\{0, \Delta - a_{rj}\}$

The resulting constraint is

$$\sum_{j\notin C_0} a_{rj}x_j + \sum_{j\in C_0} \max\{a_{rj} - \Delta,\ 0\}\, x_j \leq b - \Delta, \qquad \text{(strengthened)}$$

- **However, if the clique C is partially implied by (original), we have to add $\sum_{j\in C} x_j \leq 1$ as a constraint** [Achterberg, 2007]
- **Similar difficulties as in redundant constraint detection**

Clique Partition Algorithm with Source Attribution

Why do we need clique partition?

$$\begin{aligned} \max\quad & \textstyle\sum_{j\in\mathcal{N}} a_jx_j \\ \text{s. t.}\quad & \textstyle\sum_{j\in C_q} x_j \leq 1,\ \{C_q\}_{q=1}^{Q} \subseteq \mathcal{C},\ Q \in \mathbb{N}_+ \\ & x \in \{0,1\}^n. \end{aligned} \qquad (3)$$

- Choose disjoint cliques $\Longrightarrow$ solve (3) in $\mathcal{O}(n)$.
- Use as many cliques as possible to derive tight upper bounds.

Algorithm 1 Clique partition algorithm with source attribution

1: Initialize $V = \mathcal{N}_r$, $q = 1$, $\mathcal{C} = \emptyset$, $F = \emptyset$.
2: Set $k = 0$, $S_0 = V$, $C_q = \emptyset$, $flag$ = false.
3: Randomly choose $j \in S_k$, set $C_q = C_q \cup \{j\}$ and

$$S_{k+1} := \left\{j' \in S_k \mid x_{j'} \neq x_j,\ \exists\ \hat{C} \in \mathcal{C},\ x_{j'},\ x_j \in \mathcal{C}.\right\}.$$

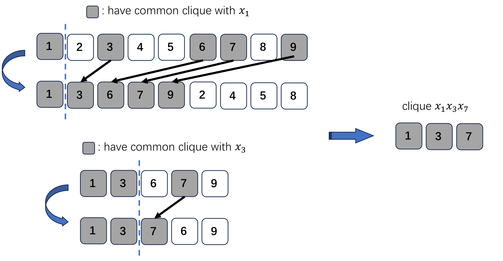
4: If $\hat{C}$ is implied by constraint r, set $flag$ = true
5: If S_{k+1} is not empty, go to step 3; otherwise C_q forms a clique. Set

$$\mathcal{C} = \mathcal{C} \cup \{C_q\},\ F = F \cup \{flag\},\ V = V \setminus C,\ q = q + 1,$$

and go to step 2.

Clique Partition: An Example

Perform clique partition for a constraint of 9 binary variables:



▷ We obtain the first clique $\{x_1, x_3, x_7\}$.

Clique Enhanced Presolve

Clique + Domain Propagation (DP)

- Solve

$$\begin{aligned} \max\quad & z := b - \textstyle\sum_{j\neq j_0} a_{rj}x_j \\ \text{s. t.}\quad & \textstyle\sum_{j\in C_q} x_j \leq 1,\ \{C_q\}_{q=1}^{Q} \subseteq \mathcal{C},\ Q \in \mathbb{N}_+ \end{aligned}$$

by algorithm 1 using the full clique table $\mathcal{C}$, obtaining an upper approximation $\hat{z} \geq z$.

- Calculate upper implied bounds

$$x_{j_0} \leq \hat{z} \ / \ a_{j_0}.$$

Clique + Redundant Constraint Detection (RCD)

Lemma: If constraint r is redundant, there is at most **one** clique in the partition with source r.

- Solve (2) by algorithm 1 using the partial clique table $\mathcal{C}_r$.

If $\hat{z} \leq b_r$,

▷ if a clique C_{q_0} is implied by constraint r, replace original inequality with $\sum_{j\in C_{q_0}} x_j \leq 1$;

▷ otherwise, remove the r-th constraint.

Clique + Coefficient Strengthening (CS)

- Derive an upper bound for $\sup\left\{\sum_{j\notin C_0} a_{rj}x_j\right\}$ and check if assumptions ($\star$) hold.
- Perform coefficient strengthening for cliques that are **NOT** implied by the constraint itself.
- Re-extract cliques of this constraint if reductions are found.

Numerical Experiments

Experiment settings

- Hardware: a cluster of Intel®R Xeon®R Gold 6140R CPU at 2.30GHz
- MIP solver: HiGHS 1.9.0 [Huangfu and Hall, 2018]
- Time limit: 7200 seconds
- Workflow
  - ▷ HiGHS presolve
  - ▷ Clique augmented presolve
  - ▷ HiGHS solve
- Notations
  - ▷ T: ratio of average CPU time
    - $\ast$ < 1 indicates improvement
  - ▷ N: ratio of average node number
  - ▷ #S: the number of instances solved by a setting
  - ▷ #Ins: the number of instances solved by at least one setting

Default v.s. Clique + DP

	Default	Clique-DP	Compare		Affected		
Bracket	#S	#S	T	N	#Ins	T	N
≥ 0	732	728	1.01	0.98	75	0.94	0.77
≥ 10	654	650	1.02	0.97	75	0.94	0.77
≥ 100	504	500	1.02	0.97	65	0.93	0.78
≥ 1000	253	249	1.01	0.96	50	0.95	0.77

Table 1: Performance of using clique augmented domain propagation.

Default v.s. Clique + RCD

	Default	Clique-DP	Compare		Affected		
Bracket	#S	#S	T	N	#Ins	T	N
≥ 0	732	727	1.01	0.96	106	1.06	0.74
≥ 10	653	648	1.01	0.96	91	1.08	0.71
≥ 100	504	499	1.01	0.94	74	1.08	0.66
≥ 1000	251	246	1.01	0.94	32	1.08	0.56

Table 2: Performance of using clique augmented redundant constraint detection.

Default v.s. Clique + CS

	Default	Clique-DP	Compare		Affected		
Bracket	#S	#S	T	N	#Ins	T	N
≥ 0	726	724	0.95	0.94	121	0.75	0.64
≥ 10	647	645	0.95	0.93	106	0.73	0.59
≥ 100	498	496	0.95	0.91	84	0.75	0.51
≥ 1000	252	250	0.93	0.86	44	0.64	0.32

Table 3: Performance of using clique augmented coefficient strengthening.

Conclusion & Reference

Conclusion

- We proposed a clique partition algorithm with source attribution to enhance domain propagation, redundant constraint detection and coefficient strengthening.
- The three proposed clique based presolve techniques can notably improve the performance of open source solver HiGHS.

Reference

Achterberg, T. (2007). *Constraint Integer Programming*. PhD thesis, Technische Universiat Berlin.

Achterberg, T., Bixby, R. E., Gu, Z., Rothberg, E., and Weninger, D. (2019). Presolve reductions in mixed integer programming. *INFORMS Journal on Computing*, 32(2):473–506.

Borndörfer, R. (1998). *Aspects of Set Packing, Partitioning, and Covering*. PhD thesis, Technische Universitat Berlin.

Fügenschuh, A. and Martin, A. (2005). Computational integer programming and cutting planes. *Handbooks in Operations Research and Management Science*, 12(C):69–121.

Huangfu, Q. and Hall, J. A. (2018). Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142.

Savelsbergh, M. W. P. (1994). Preprocessing and probing techniques for mixed integer programming problems. *ORSA Journal on Computing*, 6(4):445–454.